

Agentic AI

**Workshop: Single Agents, Multi-Agent
Systems & Guardrails**



Learning Outcomes

Upon completion of this session, the learners should be able to:

- Explain what Agentic AI is and how it differs from a standard LLM application.
- Understand the technical design of a single-agent system, including the perceive-reason-plan-act-reflect loop and tool/function-calling architecture.
- Explain the principles and benefits of multi-agent systems, including role specialization, task decomposition, communication protocols, and coordination strategies.
- Implement single- and multi-agent workflows using modern frameworks (e.g. LangGraph, CrewAI, AutoGen, LlamaIndex) and select the right agentic design pattern for a given use case.
- Apply guardrails and human-in-the-loop controls to keep agentic systems safe, reliable, and compliant.

Workshop Agenda (3 Hours)

A hands-on tour from single-agent foundations to safe multi-agent orchestration:

- **Foundations & Single-Agent Systems** — why agents, single-agent design, tool/function calling **(40 min)**
- **Break 1** **(15 min)**
- **Multi-Agent Systems & Frameworks** — planning, communication, CrewAI/LangGraph/AutoGen/LlamaIndex, orchestration **(40 min)**
- **Use Case 1: OralLink Dental Chatbot** — architecture & discussion **(15 min)**
- **Break 2** **(10 min)**
- **Agentic AI Design Patterns & Guardrails** — choosing a pattern, keeping it safe **(35 min)**
- **Use Case 2: Procurement Exception Commander** — architecture & discussion **(15 min)**
- **Key Takeaways & Q&A** **(10 min)**

Why we need to have Agents?

Complex decision-making:

Workflows involving nuanced judgment, exceptions, or context-sensitive decisions, for example refund approval in customer service workflows.

Difficult-to-maintain rules:

Systems that have become unwieldy due to extensive and intricate rulesets, making updates costly or error-prone, for example performing vendor security reviews.

Heavy reliance on unstructured data:

Scenarios that involve interpreting natural language, extracting meaning from documents, or interacting with users conversationally, for example processing a home insurance claim.

Before committing to building an agent, validate that your use case can meet these criteria clearly

Agentic AI vs. generative AI

Agentic AI and generative AI share a foundation but serve different purposes. **Generative AI focuses on producing new content based on patterns in data.** It helps write text, create images, or suggest ideas by responding directly to prompts. Once it delivers a result, its job is done until the next prompt comes along.

Agentic AI builds on these generative abilities but has more independence to execute on goal-oriented action. An AI agent doesn't stop after generating a single output in response to a prompt—it continues to iterate on its own outputs to carry out key tasks and learn from the results. While generative AI helps create, agentic AI helps act. This distinction shapes how each fits into daily work.

Overview of Agentic AI

Agentic AI refers to AI systems that act autonomously, not just passively respond to user prompts. Unlike simple LLM-based chat apps, agentic AI systems:

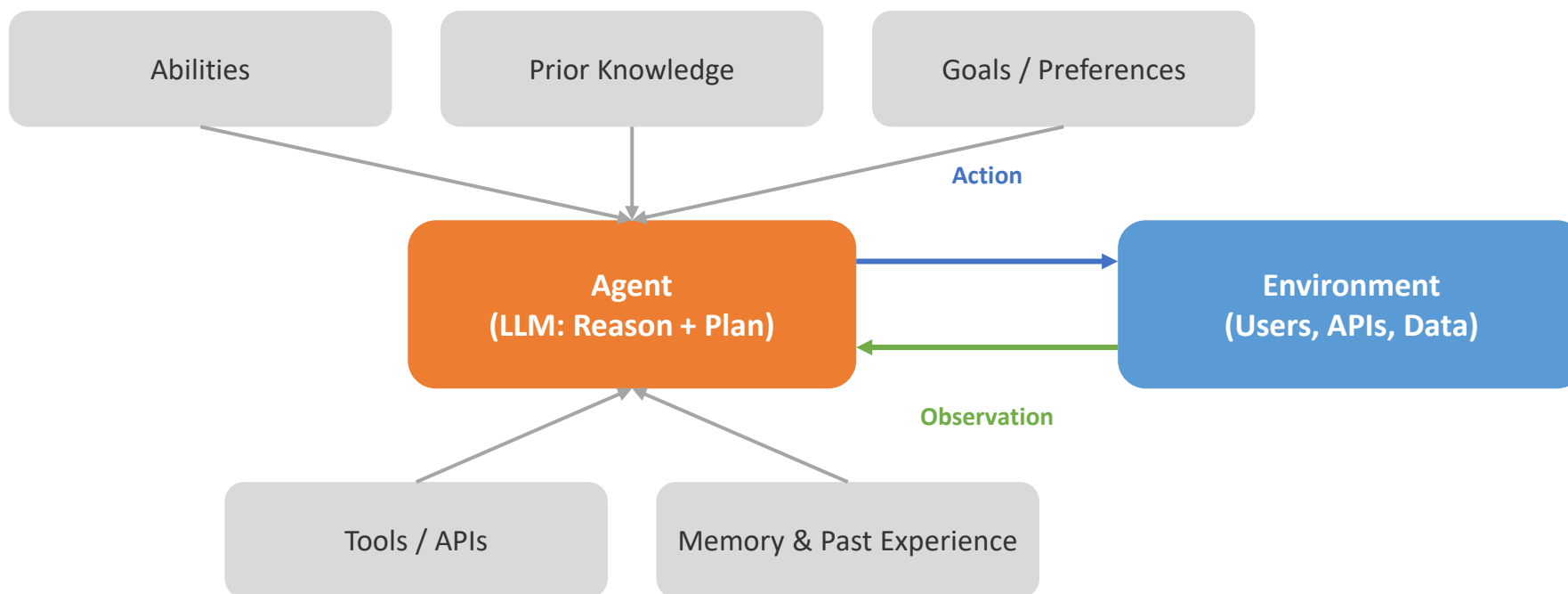
- **Perceive** the environment (input data, tools, APIs).
- **Reason** about objectives and constraints.
- **Plan** steps toward achieving a goal.
- **Act** by executing tasks, using tools, and gathering more information if needed.

A simple representation:



Overview of Agentic AI

- Anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.



Overview of Agentic AI

Key Concepts

- **Autonomy:** Executes tasks with minimal human intervention.
- **Tool Use:** Can call APIs, run code, or control external software.
- **Memory:** Maintains context across multiple steps or tasks.
- **Reflection:** Learns from mistakes to improve performance.
- **Goal-Oriented:** Works toward an outcome rather than just answering questions.

Example

- **LLM Application:** A chatbot answers FAQs about a product.
- **Agentic AI:** The same chatbot can autonomously check the inventory database, send emails, and schedule deliveries after understanding user intent.

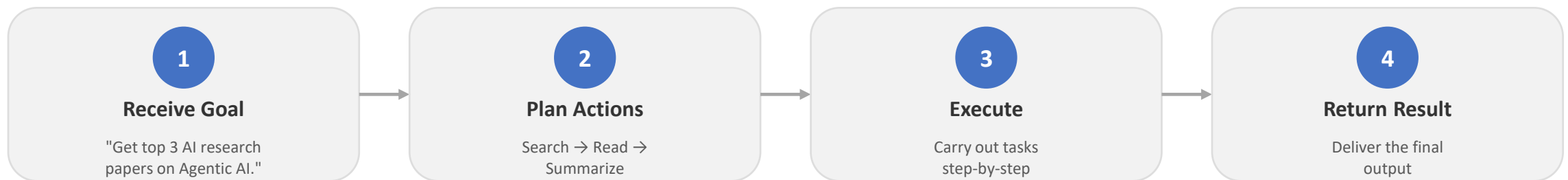
How Agentic AI Differs from Traditional LLM Applications

Feature / Aspect	LLM Application (e.g., ChatGPT API)	Agentic AI (e.g., Auto-GPT, LangGraph)
Control Flow	User-driven prompts → one response	Goal-driven → multiple steps until goal completion
State Management	Mostly stateless conversations	Persistent memory & environment interaction
Planning & Reasoning	Minimal reasoning per prompt	Iterative reasoning + task planning
Tool Use	Limited (via plugins or custom code)	Native ability to use external APIs/tools
Autonomy	Dependent on user input every step	Self-directed task execution
Error Handling	Responds as per prompt	Self-corrects using feedback loops
Applications	Q&A bots, summarization	Autonomous research assistants, workflow automation

Introduction to a Single-Agent System

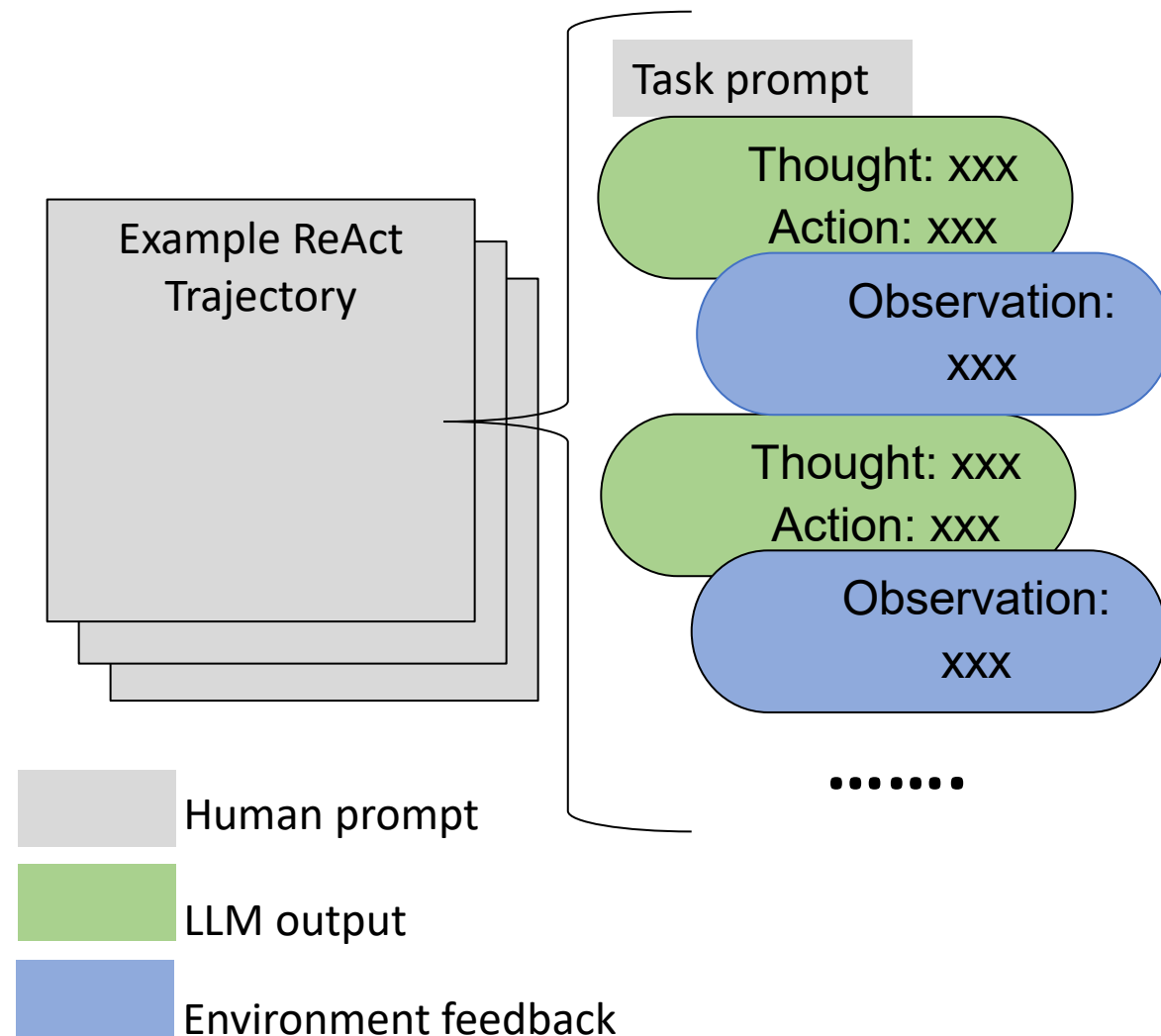
A **single agent** is the basic building block of agentic AI.
It's like one “worker” that:

The steps below show what happens once that agent is triggered:



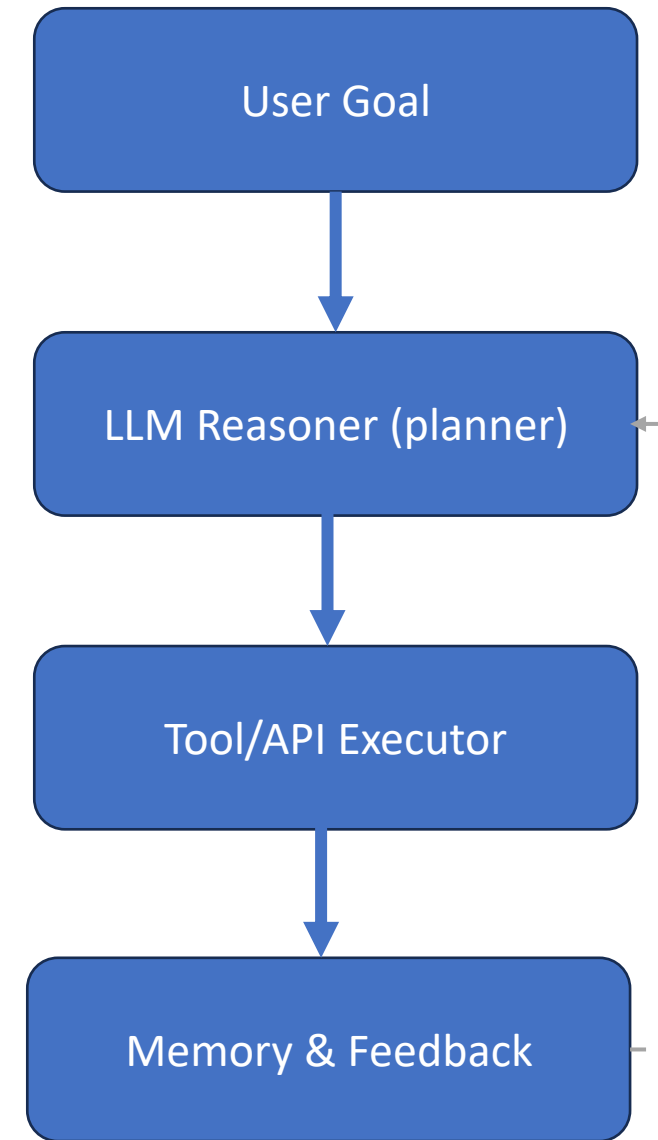
ReAct LLM-agent

- The ReAct Agent is a framework combining reasoning and action to enhance the capabilities of Large Language Models (LLMs) in solving complex tasks. It integrates reasoning (thinking through a problem) and action (interacting with external tools or environments) into a unified process, enabling the agent to work more effectively in real-world applications.



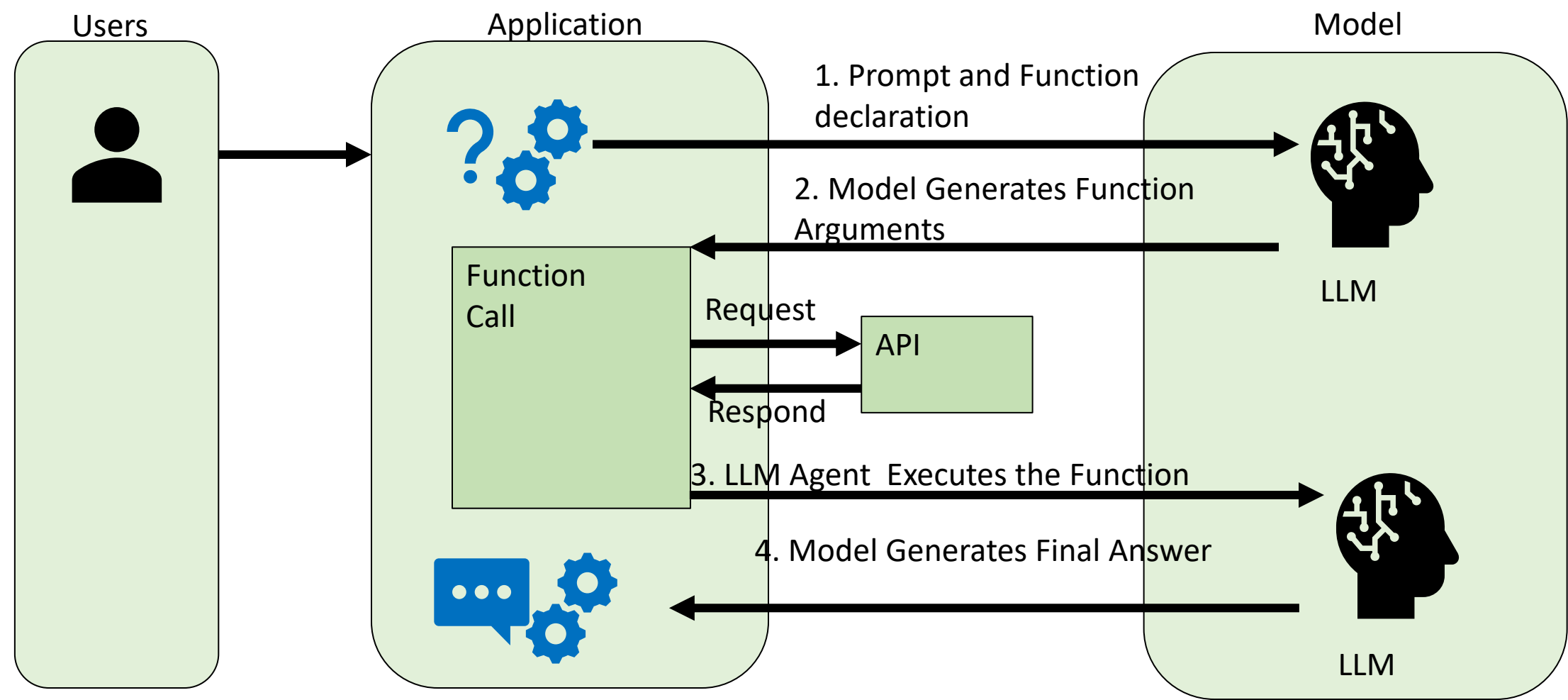
Technical Architecture

- A single-agent system typically includes:
- **LLM Reasoner**: Handles natural language understanding & reasoning.
- **Memory Store**: Vector DB (e.g., Chroma, FAISS) for contextual memory.
- **Tool Interfaces**: APIs, Python functions, external apps.
- **Planner/Executor**: Logic for task decomposition & execution.
- **Feedback Loop**: For self-correction and refinement.



--- feedback to planner

How Custom Function Calls Work



Key Benefits of Agentic AI

- **Increases efficiency and scalability:** an agent manages repetitive or time-consuming tasks with minimal supervision, freeing capacity for higher-value work.
- **Automates complex workflows autonomously:** unlike simple automation, an agent adapts to changing inputs, coordinates multiple steps, and completes workflows end to end (e.g. code modernization, refactoring).
- **Responds quickly to changing conditions:** observes what's happening and updates actions in real time as priorities shift or new data appears.
- **Frees people for high-value work:** handing off routine, multi-step tasks gives teams more time for complex problems and innovation.

Break — 15 minutes

Stretch, grab a coffee, and jot down any questions on single-agent design or tool/function calling.

We resume with Multi-Agent Systems & Frameworks.

What is a Generative AI MultiAgent?

Why Use Multiple Agents?

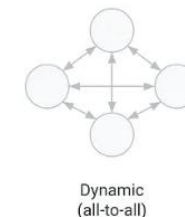
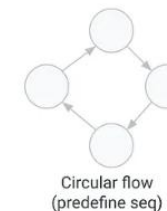
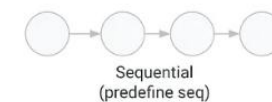
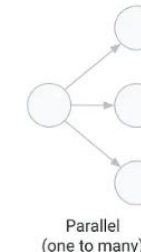
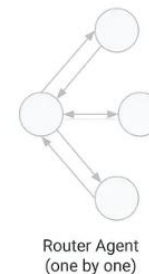
- **Single-agent limitation:** One LLM agent trying to solve everything often struggles with:
 - Context overload (too many instructions at once).
 - Weak specialization (generalist vs. expert).
 - Lack of modularity (hard to scale or debug).
- **Multi-agent benefits:**
 - **Role specialization** → different agents act as experts (e.g., planner, coder, reviewer).
 - **Parallelism** → tasks can be split among agents for faster execution.
 - **Modularity** → easier to upgrade or swap out one agent without redesigning the whole system.
 - **Human-like teamwork** → mirrors real-world collaboration.

Multi-Agent Planning & Task Decomposition

- **Hierarchical breakdown:**
 - Planner agent → breaks down the big problem into steps.
 - Worker agents → execute individual tasks.
 - Reviewer agent → checks correctness and quality.
- **Decomposition example: Build a website with product recommendations**
 - Planner: Defines steps → Data collection → Recommendation engine → Web design.
 - Data Agent: Retrieves product info.
 - ML Agent: Builds recommendation model.
 - UI Agent: Designs frontend.
 - Reviewer: Validates correctness.

Multiagent Pattern

- In the Router pattern, the router agent is using other agents as specialized tools, calling them one by one as needed.
- In the Parallel pattern, the initial agent is using multiple agents as tools simultaneously to speed up the process.
- In the Sequential and Circular Flow patterns, agents are used as tools in a predefined pipeline or loop.
- In the Dynamic pattern, agents are interacting more like a team, with each agent acting as both a user and a tool for other agents, depending on the situation.
- These patterns provide different ways to structure multi-agent interactions, allowing developers to choose the most appropriate approach based on the specific requirements of their application



Message Passing & Communication Protocols

- **Direct messaging:** Agent A sends structured text to Agent B.
- **Shared memory:** Agents post updates in a shared space.
- **Protocols:**
 - JSON-based structured messages
 - Tool invocation signals (like OpenAI's function calling)
 - Pub/Sub messaging (subscribe to updates on a task)
- **Key challenges:**
 - Ensuring clarity (avoid ambiguous prompts).
 - Preventing infinite loops (agents endlessly asking each other).
 - Controlling cost (token usage increases with more agents).

Frameworks & Tools Overview

- **CrewAI**: organizes **teams of role-based agents** (e.g. Researcher, Analyst, Writer, Critic) with task delegation and team workflows. Example: generate a market report with multiple specialized agents.
- **LangGraph**: models agent workflows as a **state graph** — nodes are agents/functions, edges are transitions — for deterministic orchestration. Example: a chatbot switching Sales → Technical → Escalation agent by intent.
- **AutoGen** (Microsoft): a **multi-agent conversation** framework — agents collaborate (e.g. Coder ↔ Reviewer loop), call tools autonomously, and can escalate to a human via its GroupChat pattern.
- **LlamaIndex Agent**: builds **data-aware agents** using vector/keyword/graph indexes for memory and grounding — works well as the "Knowledge Agent" inside a larger multi-agent workflow.

Control & Coordination

Agent Orchestration

- **Centralized:** one "controller" agent assigns tasks. **Decentralized:** agents coordinate via rules/messages, no single leader. Both aim to prevent chaos and keep the system progressing.

Shared Memory: centralized store (vector DB, blackboard), agent-specific memory (own log/history), or a hybrid — local context synced to global memory. Example: a planner keeps the global plan; workers store only their own progress.

Stopping Conditions: goal achieved and validated, time/cost budget exceeded, or deadlock detected (agents stop responding meaningfully).

Failure Recovery: retry with backoff, escalate to a human-in-the-loop, or replace the failed agent with a fallback.

Agentic AI Design Patterns

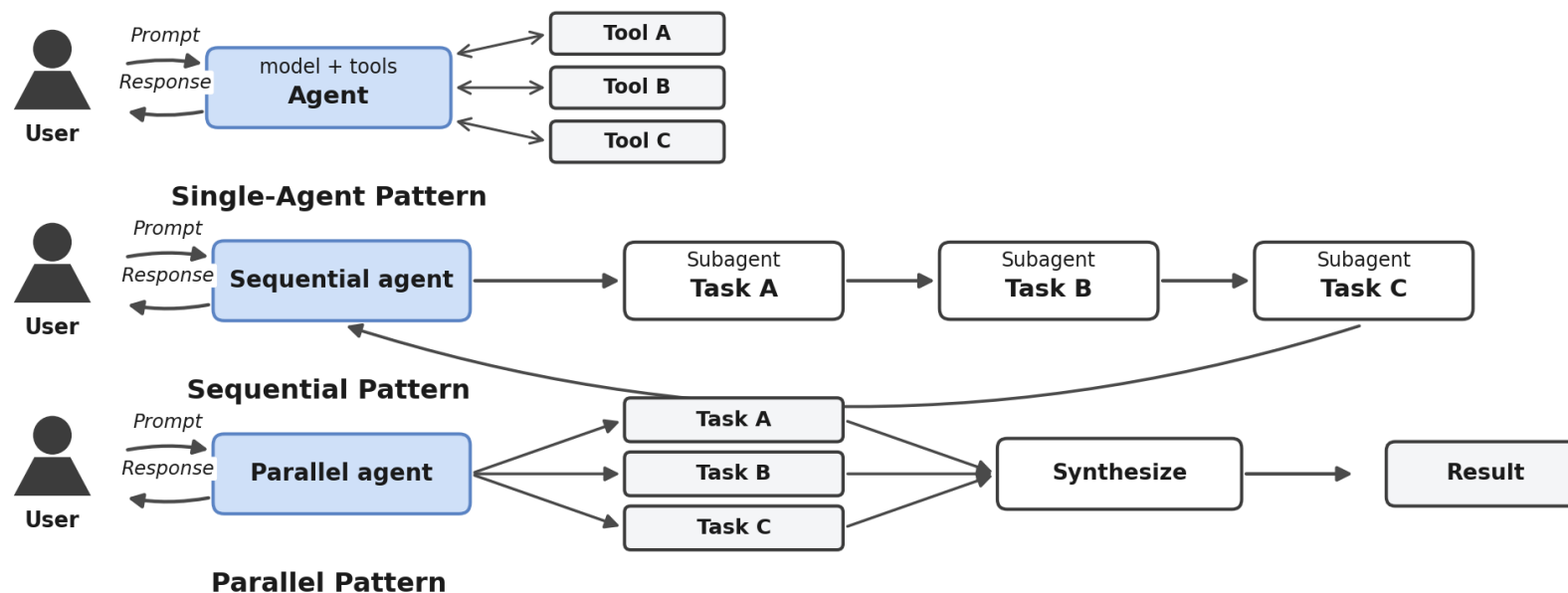
Why this matters

As agents move from prototypes to production, teams need a shared vocabulary for structuring single- and multi-agent systems (Reference: Google Cloud Architecture Center, "Choose a design pattern for your agentic AI system").

- **A 3-step framework for choosing a pattern:**
 - **1. Define requirements** → assess task complexity, latency needs, cost budget, and required human involvement.
 - **2. Review patterns** → learn the common single-agent and multi-agent approaches (next few slides).
 - **3. Select pattern** → match your workload's characteristics to the most appropriate design.
- **Key consideration:**
 - For predictable, structured tasks, a non-agentic (rule-based) solution may still be more cost-effective than building an agent.

Foundational Patterns: Single-Agent & Deterministic Workflows

- **Single-Agent Pattern:** one model with a defined tool set and system prompt autonomously plans and executes multi-step tasks. A good starting point for early-stage agent development.
- **Sequential Pattern** (deterministic): specialized agents run in a fixed, predefined order, where one agent's output becomes the next agent's input. Best for structured, repeatable pipelines, e.g. extract → clean → load.
- **Parallel Pattern** (deterministic): multiple agents work independently and simultaneously, then a synthesis step combines their outputs. Reduces latency, e.g. sentiment, keywords, category and urgency detected all at once.



Patterns Involving Iteration

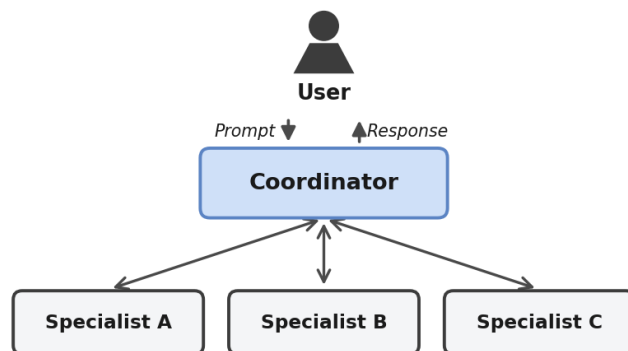
- **Loop Pattern:** subagents execute repeatedly in sequence until a termination condition is met (an iteration limit or a custom state). Good for iterative refinement or self-correction. *Risk: incorrect exit conditions can cause infinite loops and runaway cost.*
- **Review & Critique Pattern** (a loop implementation): a generator agent produces output; a critic agent evaluates it against fixed criteria and approves, rejects, or requests revision. Example: code generation checked by a security-auditor agent before approval.
- **Iterative Refinement Pattern** (a loop implementation): one or more agents progressively improve a result stored in session state until a quality threshold or iteration limit is reached. Examples: code debugging, long-form drafting.
- **ReAct Pattern** (Reason + Act): a single agent cycles through thought → action → observation until it reaches a conclusive answer, hits a max iteration, or errors out. Good for dynamic, evidence-gathering tasks, e.g. robotics path planning that avoids obstacles in real time.

Patterns Requiring Dynamic Orchestration

- **Coordinator Pattern:** a central agent uses the model to analyze a request, decompose it into sub-tasks, and dynamically dispatch each to a specialist agent at runtime. Example: a customer-service coordinator routing to order-status, return, or refund specialists.
- **Hierarchical Task Decomposition** (a coordinator implementation): agents are organized in multiple levels; a root agent decomposes a complex task, cascading down through sub-agents to worker agents. Example: a research project needing information gathering, analysis and synthesis.
- **Swarm Pattern:** agents collaborate all-to-all with no central supervisor, debating and refining a solution until an explicit exit condition (iterations, time, or goal reached). Example: market research, engineering and finance agents debating a product design trade-off.

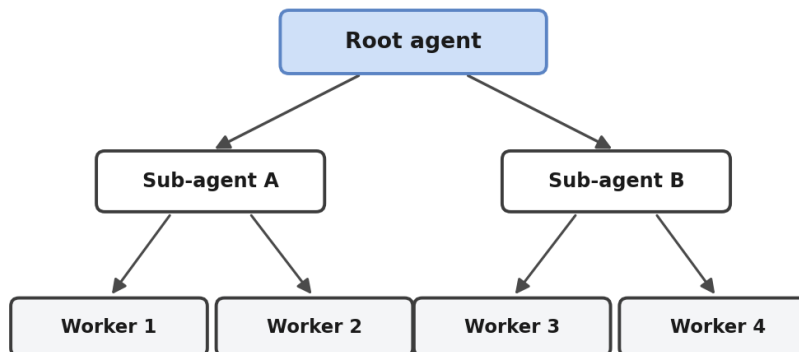
Coordinator Pattern

dynamic runtime routing



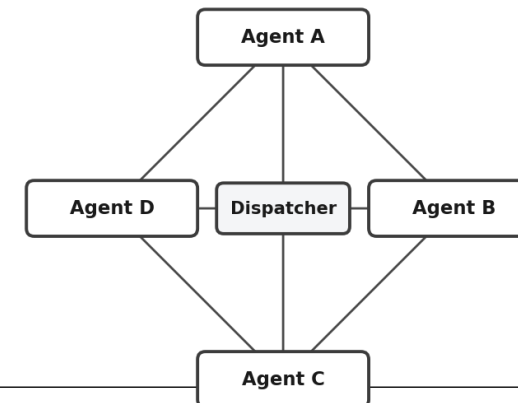
Hierarchical Task Decomposition

progressive multi-level decomposition



Swarm Pattern

all-to-all, no central supervisor



Special-Requirement Patterns

Human-in-the-Loop Pattern

- The agent workflow pauses at predefined checkpoints and calls an external system to await human review, approval, correction, or input before continuing.
 - Use when: subjective judgment, final sign-off on critical actions, or high-stakes / compliance-sensitive decisions are involved.
 - Examples: financial transaction approval, sensitive document validation, compliance officer dataset review.
 - Trade-off: improves safety and reliability, but adds the architectural complexity of a maintained external interaction system.

Custom Logic Pattern

- Maximum-flexibility, code-based orchestration using conditional logic to build complex, branching, or mixed-pattern workflows.
 - Use when: branching logic goes beyond a linear sequence, or the workflow needs fine-grained control by mixing multiple pattern types.
 - Example: a refund agent that mixes parallel verification with conditional branching to separate downstream processes

Choosing the Right Pattern: Summary

- **Deterministic workflows** (rigid, predefined paths):
 - Sequential, Parallel, Iterative Refinement.
- **Requiring dynamic orchestration** (model decides the routing at runtime):
 - Single-Agent, Coordinator, Hierarchical Task Decomposition, Swarm.
- **Involving iteration** (repeated cycles until an exit condition):
 - ReAct, Loop, Review & Critique, Iterative Refinement.
- **Special requirements** (oversight or unique branching logic):
 - Human-in-the-Loop, Custom Logic.
- *Remember: for predictable, structured tasks a simpler, non-agentic solution can still be the most cost-effective choice.*

Use Case 1: OralLink Dental Chatbot — Architecture

- One chat box, three specialist agents behind an LLM router — a **Coordinator pattern** built on CrewAI, LlamaIndex, ChromaDB, and FastAPI.
- **Router** (temperature 0): reads the 3 agent-registry descriptions plus the last 6 conversation turns and returns exactly one agent name — no keyword or regex matching.
- **Appointment Agent** (12 tools): runs the full booking lifecycle — find/verify patient, show slots, book, cancel.
- **Dental Knowledge Agent** (1 tool): RAG over ChromaDB; answers strictly from retrieved Q&A and declines clinical judgments.
- **Clinic Locator Agent** (1 tool): geocodes a Singapore postal code and ranks nearby clinics via a public government API.
- Every turn rebuilds a fresh agent instance — no shared state; continuity comes from replaying conversation history, not memory.

Use Case 1: Why This Shape? (Discussion)

- **Semantic routing beats keyword routing:** survives phrasing it has never seen and resolves short, context-only replies (e.g. just a name or a postal code).
- **Narrow roles keep tool selection accurate:** no single agent juggles all tools across domains, which shrinks its prompt and its chance of picking the wrong tool.
- **Adding a specialist is additive:** a new agent, its tools, and one registry entry — no existing agent or routing logic is touched.
- **Answers are grounded, not recalled:** tool output bounds what each agent is allowed to say.
- **Discuss:** which design pattern from this workshop does OralLink map to — and where would you draw the line between “one more specialist” and “too many agents to route between”?

Break — 10 minutes

Quick stretch before we look at Agentic AI Design Patterns and Guardrails.
We'll close with a second use case: Procurement Exception Commander.

Guardrails for Agentic AI

Autonomy plus tool access means mistakes can execute, not just display — safety controls must sit around and inside the agent, not only in the prompt.

- **Guardrails in tools, not just prompts:** a lookup tool should return “not found” instead of a nearest match, so an agent can't guess past missing data.
- **Bound every loop and retry:** cap iterations and re-plans (e.g. max 2 retries, 2 re-plans) and escalate instead of looping forever.
- **Human-in-the-loop gates:** route high-severity, high-value, or policy-triggered cases to a human approver before execution.
- **Config as the seam:** keep thresholds and limits in config, not code, so tuning doesn't require touching agent logic.
- **Complete audit trail:** log every stage's input, output, and decision — the trail, not the chat transcript, is the record of why an action happened.

Use Case 2: Procurement Exception Commander — Architecture

- One deterministic **Orchestrator** (plain Python, not an LLM) sequences 7 tool-using agents to turn a disruption notice into a resolved, audited procurement decision.
- **Read-only agents (1–4)**: Intake & Reconciliation → Impact Assessment || Alternative Sourcing (parallel) → Severity & Justification.
- **Reasoning-only agents (5–6)**: Recovery Plan Evaluator ranks options; Commander Approval Gate formats the human-facing card — neither holds tools.
- **Execution agent (7)**: the only agent with write tools — updates the system of record and notifies stakeholders.
- **Guardrail triggers force a human**: severity High/Critical, unresolved data flags, escalation-pattern contract clauses, exposure over \$100K, or zero viable options.

Use Case 2: Orchestrator Control Logic (Discussion)

- **Control lives in code, not judgement:** sequencing, retries, and the approval gate's six triggers are deterministic Python — no prompt drift can skip them.
- **Data exceptions never retry:** an unknown supplier or missing contract goes straight to a human on the first attempt, not after wasted retries.
- **Bounded re-plan loop:** a rejection sends feedback back to the Evaluator, capped at 2 attempts, then escalates — the system never assumes a human will eventually approve.
- **Narrow tools, narrow blast radius:** 5 agents get read-only tools, 1 gets write tools, 2 get none at all.
- **Discuss:** compare this fixed-pipeline, code-orchestrated design to OralLink's LLM-routed design — when would you choose determinism over flexibility?

Key Takeaways

- Start with the simplest pattern that meets the requirement — single-agent or even non-agentic first; add orchestration only when task complexity demands it.
- Multi-agent systems trade complexity for specialization, parallelism, and modularity — worth it when one generalist agent overloads or under-performs.
- Guardrails belong in tools, config, and control flow — not only in prompts — and every loop needs a bound and an exit.
- Real systems (OralLink, Procurement Exception Commander) combine these patterns pragmatically: route, specialize, and gate with humans where it matters.

References

- <https://www.voicespin.com/glossary/voice-activity-detection/>
- <https://www.youtube.com/watch?v=xWhl8RkRSGQ&t=60s>
- <https://docs.cloud.google.com/architecture/choose-design-pattern-agentic-ai-system>
- <https://medium.com/@yugank.aman/advanced-rag-techniques-0c283aac5ba>
- <https://www.llamaindex.ai/blog/a-cheat-sheet-and-some-recipes-for-building-advanced-rag-803a9d94c41b>
- *Use case sources: "Orallink Architecture" and "Procurement Exception Commander" system-architecture documents, provided for this workshop.*

Discussion & Q&A

- Which agentic design pattern best fits a workflow you deal with day-to-day — and why?
- Where would you insert a human-in-the-loop gate, and what would trigger it?
- What's one guardrail you'd add before letting an agent take a write action in your own systems?

Open floor for questions.